

FastAPI — Complete Overview

1. What is FastAPI?

FastAPI is a modern, high-performance web framework for building APIs using Python.

It is built on top of **Starlette** for web functionality and **Pydantic** for data validation and serialization.

FastAPI is commonly used for REST APIs, backend services, microservices, machine learning APIs, AI applications, authentication systems, and database-backed applications.

2. Why is FastAPI Popular?

High Performance

FastAPI supports asynchronous programming using Python's **async** and **await** syntax. This is especially useful for I/O-bound operations such as database calls and external API requests.

Automatic Validation

FastAPI uses Pydantic models to validate incoming data. Invalid data automatically produces a structured validation error.

Automatic API Documentation

FastAPI automatically generates interactive API documentation through Swagger UI and ReDoc.

3. Basic FastAPI Application

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello World"}
```

FastAPI() creates the application. The GET decorator defines an endpoint, and the function handles requests to that endpoint. The returned dictionary is converted to JSON.

4. API Routes

A route defines how the server responds to a particular HTTP request. FastAPI supports common HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

```
@app.get("/users")
def get_users():
    return {"users": []}

@app.post("/users")
def create_user():
    return {"message": "User created"}
```

5. Path Parameters

Path parameters are values included directly in the URL. They are useful when identifying a specific resource.

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

6. Query Parameters

Query parameters are values provided after a question mark in the URL. They are commonly used for filtering, searching, sorting, and pagination.

```
@app.get("/users")
def get_users(limit: int = 10):
    return {"limit": limit}
```

7. Request Body with Pydantic

Pydantic models define the structure of incoming JSON data and allow FastAPI to validate it automatically.

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
def create_user(user: User):
    return user
```

8. Running FastAPI

A common way to run a FastAPI application is with Uvicorn:

```
uvicorn main:app --reload
```

main refers to main.py, **app** refers to the FastAPI application object, and **--reload** automatically restarts the development server when code changes.

9. Interactive Documentation

After starting the server, FastAPI typically provides:

Swagger UI: /docs

ReDoc: /redoc

These interfaces allow developers to inspect endpoints, understand request and response formats, and test APIs directly from the browser.

10. FastAPI in an AI/RAG Project

FastAPI is particularly useful as the backend API layer for an AI or RAG application. A typical architecture is:

User → FastAPI → Document Processing → Chunking → Embeddings → Vector Database → LLM → FastAPI → User

For example, a POST /ask endpoint can receive a user's question, create an embedding for the question, search a vector database for relevant chunks, send the retrieved context to an LLM, and return the generated answer.